

FUNCTIONAL PROGRAMMING IN PYTHON

ANNOUNCEMENTS

- Midterm 2 on Monday during class time
- TA applications released now (link on Ed), due Friday, April 17th. **No late days.**
- HW7: Recursion comes out after the exam next week, due the following week
 - after that, HW8, then HW9 part 1 and part 2 and then we're done
- Final exam is on Wednesday May 6th from 12-2pm in Towne 100 (this room)
 - no makeups except the official university makeup in AUGUST
 - schedule your accommodated exams ASAP

TESTING OBJECTS

Any unit test needs:

- input
- expected output
- actual output
- some kind of assertion to compare

But what's the "input" to an object?

Imagine I have two accounts at a bank. One is savings, the other is checking.

- I want to always keep up to **k** dollars in my checking; beyond that, when I deposit, the money should go into savings.
- If I have to pay from my bank, the money should first come from checking, then from savings.

```
class Accounts:
    def __init__(self, threshold: int):
        self.checking = 0
        self.savings = 0
        self.threshold = threshold
    def deposit(self, amount: int):
        room_in_checking = self.checking - self.threshold
        self.checking += room_in_checking
        self.savings += amount - room_in_checking

    def pay(self, amount: int):
        if amount > self.checking:
            self.checking = 0
            self.savings -= amount - self.checking
        else:
            self.checking -= amount
```

(not bothering to check whether there's enough money right now—keep it simple)

How do I test that a new account starts with no money?

```
def test_new_accounts_empty_checking(self):  
    # "INPUT" is really setting up the object  
    acct = Accounts(100)  
    expected = 0  
    actual = acct.checking  
    self.assertEqual(expected, actual)
```

```
def test_new_accounts_empty_savings(self):  
    # "INPUT" is really setting up the object  
    acct = Accounts(100)  
    expected = 0  
    actual = acct.savings  
    self.assertEqual(expected, actual)
```

How do I test that a new account with a deposit < threshold goes into checking only?

```
def test_new_accounts_small_deposit_goes_checking(self):  
    # "INPUT" is really setting up the object *in multiple stages*  
    acct = Accounts(100)  
    acct.deposit(50)  
    expected = 50  
    actual = acct.checking  
    self.assertEqual(expected, actual)
```

How do I write two tests to verify that a new account with a deposit > threshold...

- fills the checking up to the threshold?
- fills the savings with the remainder?

C12

```
def test_new_accounts_large_deposit_fills_checking(self):
```

```
def test_new_accounts_large_deposit_overflow_in_savings(self):
```

answers at end of slides



STORY TIME

Consider a table modeling posts on a social network:

HANDLE	TIME	TEXT	IMAGE_URL
@alice.bsky.social	2025-10-21 09:15	Just had the best coffee! ☕	https://img.bsky.social/abc123.jpg
@bob_codes	2025-10-21 10:30	Check out my new Python project	None
@charlie.dev	2025-10-21 11:45	Does anyone know how to fix this bug?	https://img.bsky.social/def456.png
@diana_smith	2025-10-21 13:20	Beautiful sunset today 🌅	https://img.bsky.social/ghi789.jpg
@eve.tech	2025-10-21 14:05	Excited to share my latest blog post!	None

Each row of the table represents a "Post"—could model with a tuple, for example.

That's a
lot of
Posts.

Daily Posts on Bluesky

3.69M

-5.4% ↘



The number of posts created each day is down by 5.4% compared to 12 weeks ago.

```
import sys
post = Post(handle="@tomwalker.bsky.social",
            time="2023-06-16 11:57",
            text="""KOKO THE GORILLA: Koko birkin bag.
                    Practical Koko possession bag
                    RESEARCHER: No, Koko. You can't have a Birkin bag.
                    KOKO: Good Birkin good Koko give
                          beautiful Koko deserve gorilla
                    RESEARCHER: Koko, we simply can't afford a Birkin ba
                    It is an unjustifiable expense.
                    KOKO: jealousy professor""",
            image_url=None)
print(sys.getsizeof(post))
```

A post like this is about 1 KB.

A computer has room for about 16GB in **RAM**, or fast "working" memory.

That's 16,000,000 posts.

Bluesky has been around for a few years, so we're well beyond what we can process on a single machine.



Space Black



New



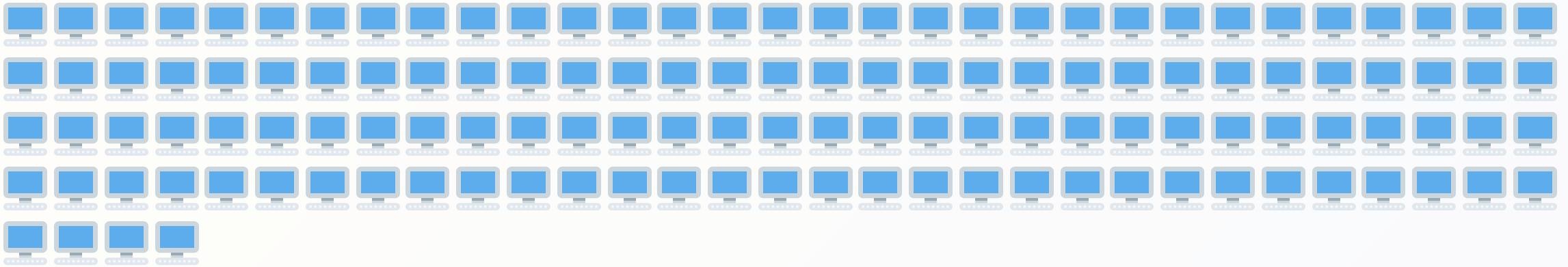
10-Core CPU

10-Core GPU

16GB Unified Memory

512GB SSD Storage¹

To host & process the posts on Bluesky, they need a lot of different computers linked up together!



Even something as simple as "find all posts by user" gets pretty complicated.

This is a fiction.

```
def find_all_posts_by_user(posts: list[Post], user: str) -> list[Post]:  
    posts_by_user = []  
    for post in posts:  
        if post.handle == user:  
            posts_by_user.append(post)  
    return posts_by_user
```

- There's no single list anywhere that stores all of the posts; it wouldn't fit on a single computer anyway.
- The same code needs to run on different machines and aggregate the results in a complicated way.

Big idea: design a system that allows a programmer to "focus on the heart of the computation," leaving the work of distributing that computation hidden:

MapReduce!

- **Map:** apply some filtering & transformation to each post
- **Reduce:** combine the filtered & transformed post into some collected answer
- Leave the details of how all posts are collected to the MapReduce implementation.

Define how you want each post to be treated, a "mapping"

```
bool map(Post p) { return p.handle == "harrysmith@bsky.social"; }
```

Define how you want to collect posts based on their mapped values

```
bool reduce(Post p, bool mapped, Set results) { if (mapped) results.add(
```

No loops, no nesting branches, just the essential core of the computation.

Programming languages of all kinds are exercises in **abstraction**.

- Programming languages in general abstract the behavior of the computer ("update this set") from the mechanics of the metal, which is just a bunch of transistors carrying current.
- Object Oriented languages allow for the expression of *real, actual things* in terms of their information properties
- Functional languages "allow the programmer to focus on the heart of the computation rather than on the details of loops, branches, and other control statements." (MIT 6.102)

REVIEW

We'll cover three general purpose higher-order functions:

- `filter`
- `map`
- `reduce` (next time)

and a new language feature: `lambda` (next time)

BEYOND SOCIAL NETWORKS...

It turns out that a LOT of problems we want to solve in computer science can be *abstracted* to one of the three functions we have shown

- `filter`
- `map` (sometimes called `transform`)
- `reduce` (sometimes called `fold`, `accumulate`, `aggregate` or other terms)

These are sort of "fundamental" patterns in computer science, showing up in many programming languages.

If you want to take more CIS courses (e.g. CIS 1200) then this is a core topic.

FILTER

`filter` is a higher order function that takes in a function and sequence and returns a new sequence containing only those elements for which the predicate function evaluates to `True`.

`filter(f, seq)` is equivalent to:

```
[elem for elem in seq if f(elem)]
```

```
res = []
for elem in seq:
    if f(elem):
        res.append(elem)
```

```
def find_posts(posts, user):  
    output = []  
    for post in posts:  
        if post.user == user:  
            output.append(post)  
    return output
```

```
def find_evens(numbers):  
    output = []  
    for num in numbers:  
        if num % 2 == 0:  
            output.append(num)  
    return output
```

```
def find_long_words(words, min_length):  
    output = []  
    for word in words:  
        if len(word) >= min_length:  
            output.append(word)  
    return output
```

```
def find_passing_grades(grades):  
    output = []  
    for grade in grades:  
        if grade >= 60:  
            output.append(grade)  
    return output
```

It's all **filter**. All of it.

If `numbers = [4, 9, 10, 13, 18, 2, 0]`

```
output = []  
for num in numbers:  
    if num % 2 == 0:  
        output.append(num)  
print(output)
```

```
def is_even(num):  
    return num % 2 == 0  
output = list(filter(is_even, numbers))  
print(output)
```

both examples print `[4, 10, 18, 2, 0]`

If `words = ["time", "cast", "a", "spell", "on", "you"]`

```
output = []
for word in words:
    if len(word) >= 4:
        output.append(word)
print(output)
```

```
def is_long(word):
    _____
output = list(filter(is_long, words))
print(output)
```

L11 What goes in the blank so that both snippets print `["time", "cast", "spell"]`?

MAP

`map` is a higher order function that takes in a function and sequence and returns a new sequence containing elements of the input sequence after having `f` applied to them.

`map(f, seq)` is equivalent to:

```
[f(elem) for elem in seq]
```

```
res = []  
for elem in seq:  
    res.append(f(elem))
```

Help! What are some ways that we could curve these scores by mapping?

```
exam_scores = [90, 89, 41, 100, 82, 71, 66]
```

PRACTICE! WHAT DO EACH OF THESE EVALUATE TO?

S7, S8, S9 If `names = ["travis", "Harry", "talie", "adi", "Sukya", "Molly"]`, what's the result?

```
def perfect_name(name):  
    return len(name) == 5 and 'a' in name  
result = list(filter(perfect_name, names))
```

```
def f(n):  
    return n[0] + n[-1]  
result = list(map(f, names))
```

```
l = filter(perfect_name, names)  
l = filter(str.islower, l)  
result = list(map(str.upper, l))
```

PRACTICE! IDENTIFYING THE PATTERN

As said before, many problems can boil down to a combination of filter/map/reduce.

For these problems what functions do you think would be needed?

(**Select all that apply for each question**, A for filter, B for map

- (M1) get a list of all the capital letters in a string
- (M2) given a list of numbers, get another list of numbers that has the squares of those numbers
- (M3) given a list of numbers, get another list of numbers that has the squares of those numbers that are less than 25

C14

Without `map/filter`: Given a list of numbers, print another list of numbers that has the squares of those numbers that are less than 25. (the squares, not the original numbers)

```
numbers = [3, -6, 8, 4, 4.1]  
output = []
```

```
print(output)
```

L13

Using `map/filter`: Given a list of numbers, print another list of numbers that has the squares of those numbers that are less than 25. (the squares, not the original numbers) *Use the helper functions defined here.*

```
def small_enough(n):  
    return n < 25  
def square(n):  
    return n * n  
numbers = [3, -6, 8, 4, 4.1]  
# TODO  
# TODO  
print(output)
```



APPLYING HIGHER ORDER FUNCTIONS TO PAST PROBLEMS

Remember the `caesar` homework?

We wrote a function called `string_to_symbol_list()` that took a string and returned a list but all characters were converted to symbols. `ord(charater) - 65`

What does this sound like?

Try implementing it with a higher order function **L15**:

```
def string_to_symbol_list(string):  
    # TODO
```

ANSWERS

```
def test_new_accounts_large_deposit_fills_checking(self):  
    # "INPUT" is really setting up the object *in multiple stages*  
    acct = Accounts(100)  
    acct.deposit(500)  
    expected = 100  
    actual = acct.checking  
    self.assertEqual(expected, actual)
```

```
def test_new_accounts_large_deposit_overflow_in_savings(self):  
    # "INPUT" is really setting up the object *in multiple stages*  
    acct = Accounts(100)  
    acct.deposit(500)  
    expected = 400  
    actual = acct.savings  
    self.assertEqual(expected, actual)
```

NEXT TIME!

REDUCE

Aggregation is done using a slightly trickier HOF called `reduce` imported from `functools`.

`reduce` is a function that takes in an *accumulator function* and a sequence and repeatedly accumulates elements from the sequence using the accumulator function.

`reduce(f, seq)` is roughly equivalent to:

```
result = seq[0]
for elem in seq[1:]:
    result = f(result, elem)
```

LAMBIDAS

Lambdas (or anonymous functions) are functions that are defined without names.

```
lambda <parameter_list> : expression
```

would become

```
def no_name(<parameter_list>):  
    return expression
```

APPLYING HOF TO CAESAR

```
def shift(symbol, n):  
    return (symbol + n) % 26 if 0 <= symbol <= 25 else symbol  
  
# assume `n` is the amount we want to shift each symbol by  
def encrypt(to_encode, n):  
    symbols = string_to_symbol_list(to_encode)  
    # TODO: put something here  
    return symbol_list_to_string(symbols)
```

Which of these would work for encrypt? (M6)

- (A) `symbols = list(map(shift, symbols))`
- (B) `symbols = list(map(lambda char : shift(char, n), symbols))`
- (C) `symbols = list(map(shift(n), symbols))`

PRACTICE CODE WRITING!

Given a list of characters, take only the ones which are singular lowercase letters and mash them up into character*3 in a string. e.g. [a,A,B,c,b,travis] becomes "aaacccbbb".

(C12)

APPLIED TO MORE COMPLEX STRUCTURES

Remember the structure we used in the check-in?

We asked you to implement the `def get_test_names(autograder_results):` which got a list of all tests in that structure.

Which higher order functions would apply here?

```
{ "score": 44.0,  
  "execution_time": 136, # optional, seconds  
  "output": "Text relevant to the entire submission", # optional  
  "output_format": "simple_format",  
  "test_output_format": "text",  
  "test_name_format": "text",  
  "visibility": "after_due_date",  
  "stdout_visibility": "visible",  
  "tests":  
  [  
    {  
      "score": 1.0,  
      "max_score": 2.0,  
      "status": "failed",  
      "name": "Name of the first test here",  
      "name_format": "text",  
      "number": "1.1",  
      "output": "test output",  
      "output_format": "text",  
      "visibility": "visible", # Optional visibility setting  
      "extra_data": {} # Optional extra data to be stored  
    },  
    # another test case  
    {  
      "score": 2.0,  
      "max_score": 2.0,  
      "status": "passed",  
      "name": "Name of the second test here",  
      "name_format": "text",  
      "number": "1.2",  
      "output": "THIS IS THE OUTPUT OF THIS TEST",  
      "output_format": "text",  
      "visibility": "visible", # Optional visibility setting  
      "extra_data": {} # Optional extra data to be stored  
    },  
    # and more test cases...  
  ],  
}
```

APPLIED TO MORE COMPLEX STRUCTURES

Remember the structure we used in the check-in?

We asked you to implement the `def get_test_names(autograder_results):` which got a list of all tests in that structure.

If we used HOF we could do:

```
def get_test_names(autograder_results):  
    return list(map(lambda test : test[
```

```
{  
  "score": 44.0,  
  "execution_time": 136, # optional, seconds  
  "output": "Text relevant to the entire submission", # optional  
  "output_format": "simple_format",  
  "test_output_format": "text",  
  "test_name_format": "text",  
  "visibility": "after_due_date",  
  "stdout_visibility": "visible",  
  "tests":  
  [  
    {  
      "score": 1.0,  
      "max_score": 2.0,  
      "status": "failed",  
      "name": "Name of the first test here",  
      "name_format": "text",  
      "number": "1.1",  
      "output": "test output",  
      "output_format": "text",  
      "visibility": "visible", # Optional visibility setting  
      "extra_data": {} # Optional extra data to be stored  
    },  
    # another test case  
    {  
      "score": 2.0,  
      "max_score": 2.0,  
      "status": "passed",  
      "name": "Name of the second test here",  
      "name_format": "text",  
      "number": "1.2",  
      "output": "THIS IS THE OUTPUT OF THIS TEST",  
      "output_format": "text",  
      "visibility": "visible", # Optional visibility setting  
      "extra_data": {} # Optional extra data to be stored  
    },  
    # and more test cases...  
  ],  
}
```

APPLIED TO MORE COMPLEX STRUCTURES

Remember the structure we used in the check-in?

We asked you to implement the `def`

`get_failing_test_names(autograder_results):`
which got a list of all failing tests in that structure.

(e.g. the status was "failed")

How would we do it with HOF?
(C14)

```
{ "score": 44.0,  
  "execution_time": 136, # optional, seconds  
  "output": "Text relevant to the entire submission", # optional  
  "output_format": "simple_format",  
  "test_output_format": "text",  
  "test_name_format": "text",  
  "visibility": "after_due_date",  
  "stdout_visibility": "visible",  
  "tests":  
  [  
    {  
      "score": 1.0,  
      "max_score": 2.0,  
      "status": "failed",  
      "name": "Name of the first test here",  
      "name_format": "text",  
      "number": "1.1",  
      "output": "test output",  
      "output_format": "text",  
      "visibility": "visible", # Optional visibility setting  
      "extra_data": {} # Optional extra data to be stored  
    },  
    # another test case  
    {  
      "score": 2.0,  
      "max_score": 2.0,  
      "status": "passed",  
      "name": "Name of the second test here",  
      "name_format": "text",  
      "number": "1.2",  
      "output": "THIS IS THE OUTPUT OF THIS TEST",  
      "output_format": "text",  
      "visibility": "visible", # Optional visibility setting  
      "extra_data": {} # Optional extra data to be stored  
    },  
    # and more test cases...  
  ],  
}
```